DSA REFERENCE

Prefix Sum

Recognition cues, invariants, code templates, and 5-8 referenced problems.

Precompute cumulative totals so any range aggregate becomes a difference of two prefix states.

VERSION 1.0 REVIEW
REVIEWED THROUGH 2026-07-18
STATUS HUMAN REVIEW REQUIRED

DSA / PREFIX-SUM

DSA

Prefix Sum

CORE IDEA

Precompute cumulative totals so any range aggregate becomes a difference of two prefix states.

Recognition cues

+ many range-sum queries
+ subarray sum target
+ balance of categories
+ immutable array aggregation

Implementation sequence

+ choose a zero-length prefix
+ define interval convention
+ combine with a frequency map for target subarrays
+ promote numeric type when sums grow

Invariant

prefix[i] represents exactly the first i elements, making range [l, r) equal prefix[r] - prefix[l].

Worked reasoning

A subarray ending at current prefix P sums to k for every earlier prefix P-k.

Complexity

O(n) preprocessing and O(1) range queries, or O(n) for target-subarray counting with a map.

Failure modes

- [ ] off-by-one ranges
- [ ] forgetting the initial zero prefix
- [ ] integer overflow
- [ ] using it for frequently updated arrays without a Fenwick tree

Practice ladder

Prefix Sum: referenced problems

Open the official problem page, classify the pattern, write the invariant, and predict the complexity before coding.

#	Problem	Deliberate-practice prompt
1	**Range Sum Query - Immutable**	State the cue, invariant, edge case, and proof before implementation.
2	**Subarray Sum Equals K**	State the cue, invariant, edge case, and proof before implementation.
3	**Contiguous Array**	State the cue, invariant, edge case, and proof before implementation.
4	**Product of Array Except Self**	State the cue, invariant, edge case, and proof before implementation.
5	**Continuous Subarray Sum**	State the cue, invariant, edge case, and proof before implementation.

Before coding

+ Pattern cue
+ State
+ Invariant
+ Boundary convention

Before finishing

+ Dry run
+ Edge case
+ Time proof
+ Space proof

PUBLISHING STATUS
REVIEW GATE: STABLE TECHNICAL. AI-produced draft. A named human must verify technical accuracy, sourcing, freshness, and editorial quality before publication. Reviewed-through date: 2026-07-18.